

Documentação SQL do Projeto Final P4

Laboratório de Bases de Dados

Bruno Kazuya Yamato Sakaji – 14562466

Julia Cavallio Orlando – 14758721

Leonardo Marangoni – 14747614

Tiago Chaves Bezerra Rocha – 14609637

1º Semestre de 2026

Sumário

1	Objetivo da documentação	5
2	Visão geral da organização SQL	5
3	Schema: tabelas do banco	6
3.1	Tabelas geográficas e auxiliares	6
3.1.1	Tabela <code>seasons</code>	6
3.1.2	Tabela <code>continents</code>	6
3.1.3	Tabela <code>countries</code>	7
3.1.4	Tabela <code>cities</code>	7
3.1.5	Tabela <code>time_zones</code>	7
3.1.6	Tabela <code>feature_codes</code>	7
3.1.7	Tabelas de idiomas	8
3.2	Tabelas de aeroportos	8
3.2.1	Tabela <code>airport_types</code>	8
3.2.2	Tabela <code>airports</code>	8
3.3	Tabelas da Fórmula 1	9
3.3.1	Tabela <code>circuits</code>	9
3.3.2	Tabela <code>constructors</code>	9
3.3.3	Tabela <code>drivers</code>	9
3.3.4	Tabela <code>status</code>	10
3.3.5	Tabela <code>racers</code>	10
3.3.6	Tabela <code>results</code>	10
3.3.7	Tabela <code>qualifying</code>	10
3.3.8	Tabela <code>standings</code>	10

3.3.9	Tabelas <code>driver_standings</code> e <code>constructor_standings</code>	11
3.4	Tabelas específicas do P4	11
3.4.1	Tabela <code>users</code>	11
3.4.2	Tabela <code>users_log</code>	11
3.4.3	Tabela <code>constructor_drivers</code>	12
4	Carga dos dados	12
4.1	Arquivos carregados	12
4.2	Uso do schema de estágio	13
4.3	Funções de conversão segura	13
4.4	Conversão da nacionalidade textual	14
4.5	Volumes validados na execução	14
5	Deduplicação e métricas do T1	15
5.1	Métricas antes da deduplicação	15
5.2	Critério de deduplicação	15
5.3	Atualização de referências	16
5.4	Métricas após a deduplicação	16
6	Visões	16
6.1	Visões inspiradas no T1	16
6.1.1	View <code>vw_t1_pontos_pilotos_ano</code>	16
6.1.2	View <code>vw_t1_poles_por_piloto</code>	16
6.1.3	View <code>vw_t1_paises_sede_corridas_cidades_aeroportos</code>	16
6.2	Visões inspiradas no T4	17
6.2.1	Visão materializada <code>Aeroportos_Brasileiros</code>	17
6.2.2	View <code>Aeroportos_sem_cidades</code>	17
6.2.3	View <code>Cidades_brasileiras</code>	17
6.2.4	View <code>Circuitos_completa</code>	17
6.2.5	View <code>Problemas_aeroportos</code>	17
6.2.6	View <code>Correcao_aeroportos</code>	17
6.3	Visões específicas do P4	17
6.3.1	View <code>vw_p4_results_full</code>	17
6.3.2	View <code>vw_p4_constructor_driver_history</code>	18
6.3.3	View <code>vw_p4_driver_latest_constructor</code>	18
6.3.4	View <code>vw_p4_cidades_brasileiras_todas</code>	18
6.3.5	View <code>vw_p4_aeroportos_brasileiros_medium_large</code>	18
7	Funções e procedimentos	19
7.1	Autenticação e log de acesso	19
7.1.1	Função <code>p4_authenticate</code>	19
7.1.2	Função <code>p4_log_access</code>	19
7.2	Funções e procedimento inspirados no T2	19

7.2.1	Função Nome_Nacionalidade	19
7.2.2	Função Pilotos_Nacionalidade	19
7.2.3	Procedimento Cidade Chamada	20
7.2.4	Função Numero_Vitorias	20
7.2.5	Função Pais_Continente	20
7.3	Dashboard do administrador	20
7.3.1	Função p4_admin_counts	20
7.3.2	Função p4_admin_latest_races	20
7.3.3	Função p4_admin_latest_constructors_points	20
7.3.4	Função p4_admin_latest_drivers_points	21
7.4	Dashboard da escuderia	21
7.4.1	Função p4_constructor_dashboard	21
7.5	Dashboard do piloto	21
7.5.1	Função p4_driver_dashboard	21
7.6	Relatórios do administrador	22
7.6.1	Relatório 1: função p4_report_admin_status_count	22
7.6.2	Relatório 2: função p4_report_admin_airports_near_city	22
7.6.3	Relatório 3: funções de escuderias e hierarquia de corridas	22
7.7	Relatórios da escuderia	23
7.7.1	Função p4_report_constructor_driver_wins	23
7.7.2	Função p4_report_constructor_status_count	23
7.8	Relatórios do piloto	24
7.8.1	Função p4_report_driver_points_by_year_race	24
7.8.2	Função p4_report_driver_status_count	24
7.9	Funções de cadastro	24
7.9.1	Função p4_create_constructor	24
7.9.2	Função p4_create_driver	24
7.9.3	Função p4_create_driver_for_constructor	24
7.9.4	Função p4_constructor_find_driver_by_family_name	25
8	Usuários iniciais	25
8.1	Administrador	25
8.2	Escuderias	25
8.3	Pilotos	26
9	Gatilhos	26
9.1	Gatilho de pilotos	26
9.2	Gatilho de escuderias	26
10	Índices	27
10.1	Índices de chaves estrangeiras	27
10.2	Índice de login	27
10.3	Índice Hash de nome completo	28

10.4	Índice B-tree parcial para cidades brasileiras	28
10.5	Índices específicos para dashboards e relatórios	28
10.6	Índices geográficos	29
11	Testes rápidos	29
12	Como o backend deve usar o banco	29
12.1	Login	30
12.2	Log de acesso	30
12.3	Dashboards	30
12.4	Relatórios	30
12.5	Cadastro de pilotos por arquivo	30
13	Decisões de projeto	31
13.1	Schema já normalizado	31
13.2	Carga com staging	31
13.3	Deduplicação conservadora	31
13.4	Views para reduzir repetição	31
13.5	Funções para centralizar regras	31
13.6	Gatilhos para manter usuários sincronizados	31
13.7	Índices voltados às consultas do projeto	31
14	Resumo dos objetos existentes	31
14.1	Tabelas	31
14.2	Views e visão materializada	32
14.3	Funções e procedimentos	32
14.4	Gatilhos	33
14.5	Índices principais	33
15	Conclusão	34

1 Objetivo da documentação

Esta documentação descreve a parte SQL do Projeto Final P4. O objetivo é explicar quais objetos existem no banco de dados, por que eles foram criados e como eles são utilizados pela aplicação.

A base foi organizada para atender ao enunciado do projeto final, que exige uma aplicação completa com login, dashboards, relatórios, manipulação de dados, uso explícito de SQL, funções, procedimentos, gatilhos, visões, índices e registros de acesso.

A documentação cobre somente objetos que existem nos scripts SQL do projeto. Quando uma parte da funcionalidade pertence ao backend, isso é indicado explicitamente.

2 Visão geral da organização SQL

A pasta de scripts SQL está organizada da seguinte forma:

```
1 sql/  
2   01_schema_tabelas.sql  
3   02_carga_csv.sql  
4   03_t1_deduplicacao_metricas.sql  
5   04_views.sql  
6   05_functions.sql  
7   06_seed_users.sql  
8   07_triggers.sql  
9   08_indices.sql  
10  09_testes_rapidos.sql  
11  99_run_all.sql
```

Cada arquivo tem uma responsabilidade única:

- `01_schema_tabelas.sql`: cria todas as tabelas do banco.
- `02_carga_csv.sql`: carrega os dados dos arquivos CSV e TSV.
- `03_t1_deduplicacao_metricas.sql`: registra métricas e aplica deduplicação conservadora de cidades.
- `04_views.sql`: cria visões e uma visão materializada.
- `05_functions.sql`: cria funções e procedimentos usados pela aplicação.
- `06_seed_users.sql`: cria os usuários iniciais da aplicação.
- `07_triggers.sql`: cria gatilhos para sincronizar pilotos e escuderias com usuários.
- `08_indices.sql`: cria índices para melhorar o desempenho das consultas.
- `09_testes_rapidos.sql`: executa consultas de teste para validar a base.

- `99_run_all.sql`: executa todos os scripts na ordem correta.

O arquivo principal de execução é `99_run_all.sql`. Ele chama os demais scripts usando comandos do `psql`. A ordem é importante porque as funções dependem de tabelas e visões já existentes, e os índices são criados depois da carga para não prejudicar as inserções em massa.

Exemplo de execução:

```
1 psql -U postgres -d f1db_grupo5 -f 99_run_all.sql
```

3 Schema: tabelas do banco

O arquivo `01_schema_tabelas.sql` cria todas as tabelas do banco. Ele também remove tabelas antigas antes da criação, usando `DROP TABLE IF EXISTS`, para permitir recriar a base do zero durante testes.

O schema foi criado já no formato normalizado usado no P4. Isso significa que as tabelas de pilotos e escuderias não armazenam nacionalidade textual diretamente. Em vez disso, elas possuem `country_id`, que referencia a tabela `countries`. Essa decisão foi tomada para seguir a normalização trabalhada no T1 e evitar repetição de textos de nacionalidade.

3.1 Tabelas geográficas e auxiliares

3.1.1 Tabela `seasons`

Armazena as temporadas da Fórmula 1. Possui:

- `id`: identificador da temporada;
- `year`: ano da temporada, com restrição de unicidade.

Ela é usada por `racers` e por `standings` para associar corridas e classificações a um ano.

3.1.2 Tabela `continents`

Armazena continentes. Possui:

- `id`;
- `code`;
- `name`.

É usada por `countries` para indicar o continente de cada país.

3.1.3 Tabela **countries**

Armazena países. Possui:

- `id`;
- `code`;
- `name`;
- `wikipedia_link`;
- `keywords`;
- `continent_id`;
- `nationality`.

O campo `nationality` é importante porque concentra a nacionalidade que antes poderia aparecer repetida em pilotos e escuderias. Assim, `drivers` e `constructors` apontam para `countries` por meio de `country_id`.

Exemplo ilustrativo: em vez de armazenar a palavra `British` repetida para vários pilotos e escuderias, o banco armazena o país uma vez em `countries` e faz referência a ele.

3.1.4 Tabela **cities**

Armazena cidades. Possui nome, nome ASCII, nomes alternativos, coordenadas, país, fuso horário, população, elevação e data de modificação.

Essa tabela é usada para:

- vincular aeroportos a cidades;
- vincular circuitos a cidades;
- consultar cidades brasileiras no relatório de aeroportos próximos;
- apoiar a deduplicação do T1.

3.1.5 Tabela **time_zones**

Armazena fusos horários com `gmt_offset`, `dst_offset` e `raw_offset`. Esses campos são numéricos porque os arquivos TSV podem conter valores decimais.

3.1.6 Tabela **feature_codes**

Armazena códigos de características geográficas vindos da base GeoNames. A tabela é usada por `cities` por meio de `feature_code_id`.

3.1.7 Tabelas de idiomas

Foram criadas:

- `language_names`;
- `iso_language_codes`;
- `country_languages`.

Essas tabelas organizam nomes e códigos de idiomas. A coluna `iso_639_2` foi definida como `VARCHAR(20)` porque o arquivo original contém valores compostos, por exemplo códigos alternativos separados por barra.

3.2 Tabelas de aeroportos

3.2.1 Tabela `airport_types`

Armazena os tipos de aeroportos, como `small_airport`, `medium_airport` e `large_airport`. Ela evita repetir o texto do tipo diretamente em cada aeroporto.

3.2.2 Tabela `airports`

Armazena aeroportos. Os principais campos são:

- `id`;
- `ident`;
- `airport_type_id`;
- `name`;
- `latitude_deg`;
- `longitude_deg`;
- `elevation_ft`;
- `city_id`;
- `icao_code`;
- `iata_code`;
- `gps_code`;
- `local_code`;
- `home_link`;

- `wikipedia_link`;
- `keywords`.

Essa tabela é usada principalmente no Relatório 2 do administrador, que busca aeroportos brasileiros de tipo médio ou grande próximos de uma cidade brasileira.

3.3 Tabelas da Fórmula 1

3.3.1 Tabela `circuits`

Armazena circuitos. Possui referência textual, nome, latitude, longitude, cidade associada e URL da Wikipédia.

O campo de longitude no SQL é chamado `long`. Como `long` pode ser palavra problemática em alguns contextos, o schema usa aspas duplas quando necessário.

3.3.2 Tabela `constructors`

Armazena escuderias. Possui:

- `id`;
- `constructor_ref`;
- `name`;
- `country_id`;
- `wikipedia_url`.

O campo `constructor_ref` é usado para gerar o login da escuderia. Por exemplo, uma escuderia com `constructor_ref` igual a `mclaren` gera o login `mclaren_c`.

3.3.3 Tabela `drivers`

Armazena pilotos. Possui:

- `id`;
- `driver_ref`;
- `given_name`;
- `family_name`;
- `date_of_birth`;
- `country_id`.

O campo `driver_ref` é usado para gerar o login do piloto. Por exemplo, um piloto com `driver_ref` igual a `hamilton` gera o login `hamilton_d`.

3.3.4 Tabela **status**

Armazena os status dos resultados, como chegada normal, abandono, acidente, problemas mecânicos e outros status da base.

3.3.5 Tabela **racers**

Armazena corridas. Possui temporada, rodada, nome da corrida, data, horário e circuito.

Essa tabela é essencial para dashboards e relatórios por temporada, circuito e ano.

3.3.6 Tabela **results**

Armazena os resultados dos pilotos em corridas. Possui:

- corrida;
- piloto;
- escuderia;
- posição de largada;
- posição textual;
- ordem de chegada;
- pontos;
- voltas;
- status.

A coluna **position** foi definida como texto porque os dados reais podem conter tanto números quanto códigos textuais. A coluna usada para identificar vitória é **position_order**, que é numérica. Assim, uma vitória é identificada por:

```
1 position_order = 1
```

3.3.7 Tabela **qualifying**

Armazena resultados de classificação, incluindo corrida, piloto, escuderia, posição e tempos de Q1, Q2 e Q3.

3.3.8 Tabela **standings**

Armazena classificações por temporada e rodada. Possui posição, pontos e vitórias. É usada em conjunto com **driver_standings** e **constructor_standings**.

3.3.9 Tabelas `driver_standings` e `constructor_standings`

Essas tabelas fazem a ligação entre `standings` e pilotos ou escuderias. Elas possuem chave primária composta.

3.4 Tabelas específicas do P4

3.4.1 Tabela `users`

A tabela `users` armazena os usuários da aplicação. Ela possui:

- `userid`;
- `login`;
- `password`;
- `tipo`;
- `id_original`;
- `created_at`;
- `updated_at`.

A coluna `tipo` aceita apenas os valores `Admin`, `Escuderia` e `Piloto`. O campo `id_original` guarda o identificador da escuderia ou do piloto correspondente. Para o administrador, esse valor deve ser nulo.

Foi criada uma restrição para garantir essa regra:

- usuário `Admin`: `id_original` nulo;
- usuário `Escuderia` ou `Piloto`: `id_original` não nulo.

As senhas são armazenadas como hash usando a extensão `pgcrypto`.

3.4.2 Tabela `users_log`

A tabela `users_log` registra eventos de acesso ao sistema. Ela possui:

- `id`;
- `userid`;
- `action_type`;
- `action_timestamp`.

A coluna `action_type` aceita apenas `LOGIN` e `LOGOUT`. Essa tabela atende ao requisito de auditoria de login e logout.

3.4.3 Tabela `constructor_drivers`

A tabela `constructor_drivers` registra associações explícitas entre escuderias e pilotos. Ela possui:

- `constructor_id`;
- `driver_id`;
- `registered_at`.

Essa tabela foi criada para apoiar o fluxo em que uma escuderia insere novos pilotos por arquivo. O histórico oficial de pilotos que correram por uma escuderia continua sendo obtido pela tabela `results`. A tabela `constructor_drivers` é usada apenas para registrar associações criadas pelo sistema quando novos pilotos são cadastrados pela escuderia.

4 Carga dos dados

O arquivo `02_carga_csv.sql` faz a carga dos arquivos da pasta `data`. Ele utiliza arquivos CSV e TSV.

4.1 Arquivos carregados

Os arquivos usados são:

- `countries.csv`;
- `timeZones.tsv`;
- `featureCodes_en.tsv`;
- `iso-languagecodes.tsv`;
- `cities.tsv`;
- `airports.csv`;
- `circuits.csv`;
- `constructors.csv`;
- `drivers.csv`;
- `racers.csv`;
- `results.csv`;
- `qualifying.csv`;
- `driver_standings.csv`;
- `constructor_standings.csv`.

4.2 Uso do schema de estágio

A carga não insere os arquivos diretamente nas tabelas finais. Primeiro, os dados são carregados em tabelas do schema `stg`. Isso foi feito porque os arquivos originais podem conter campos vazios, valores textuais em colunas numéricas, horários com sufixos e inteiros representados como decimais.

A estratégia é:

1. criar o schema `stg`;
2. criar tabelas brutas com colunas textuais;
3. carregar os arquivos com `\copy`;
4. converter e tratar os dados;
5. inserir nas tabelas finais.

Exemplo ilustrativo:

```
1 \copy stg.drivers_raw FROM '../data/drivers.csv' WITH (FORMAT csv
  , HEADER true);
```

4.3 Funções de conversão segura

Dentro do schema `stg`, foram criadas funções para conversão segura:

- `stg.safe_text`;
- `stg.safe_numeric`;
- `stg.safe_int`;
- `stg.safe_bigint`;
- `stg.safe_smallint`;
- `stg.safe_date`;
- `stg.safe_time`;
- `stg.safe_minutes`.

Essas funções evitam que a carga quebre ao encontrar valores como campos vazios, NULL, números como 1.0 ou horários com Z. Quando um valor não pode ser convertido corretamente, ele é tratado como nulo, em vez de interromper todo o script.

4.4 Conversão da nacionalidade textual

Os arquivos originais de pilotos e escuderias possuem nacionalidade textual. Como o schema final usa `country_id`, a carga usa essa nacionalidade apenas para encontrar o país correspondente em `countries`.

Exemplo ilustrativo:

```
1 INSERT INTO drivers (id, driver_ref, given_name, family_name,
2   date_of_birth, country_id)
3 SELECT
4     stg.safe_int(driver_id),
5     stg.safe_text(driver_ref),
6     stg.safe_text(given_name),
7     stg.safe_text(family_name),
8     stg.safe_date(date_of_birth),
9     c.id
10 FROM stg.drivers_raw d
11 LEFT JOIN countries c ON c.nationality = d.nationality;
```

Esse exemplo mostra a ideia principal: a nacionalidade textual não é mantida em `drivers`; ela é convertida para uma chave estrangeira.

4.5 Volumes validados na execução

A execução validada carregou os dados e exibiu, entre outros, os seguintes totais:

- 249 registros carregados inicialmente de países;
- 418 registros de fusos horários;
- 685 registros de códigos de características;
- 7929 registros de códigos de idioma;
- 33430 cidades carregadas antes da deduplicação;
- 84958 aeroportos;
- 168 escuderias;
- 616 pilotos;
- 1149 corridas;
- 7600 resultados;
- 3016 registros de classificação;
- 3131 registros de classificação de pilotos;
- 1061 registros de classificação de escuderias.

5 Deduplicação e métricas do T1

O arquivo `03_t1_deduplicacao_metricas.sql` aplica uma deduplicação conservadora de cidades e registra métricas antes e depois do processo.

5.1 Métricas antes da deduplicação

A tabela `t1_metrics_before` guarda contagens gerais antes da deduplicação:

- países;
- pilotos;
- escuderias;
- cidades;
- aeroportos;
- circuitos.

A tabela `t1_country_reference_metrics` verifica quantos pilotos e escuderias ficaram sem `country_id` após a carga.

5.2 Critério de deduplicação

A tabela `t1_city_dedup_map` identifica cidades duplicadas usando um critério conservador:

- mesmo país;
- mesmo nome normalizado, usando `ascii_name` quando disponível;
- manutenção preferencial do registro com maior população;
- em caso de empate, preferência para registros com coordenadas;
- em novo empate, menor `id`.

Esse critério foi escolhido por ser simples de explicar e menos arriscado do que unificar cidades apenas por proximidade geográfica. Ele reduz duplicatas evidentes sem alterar agressivamente a base.

5.3 Atualização de referências

Depois de criar o mapa de deduplicação, o script atualiza as chaves estrangeiras em:

- `airports`;
- `circuits`.

Em seguida, remove da tabela `cities` os registros duplicados que já foram substituídos por registros mantidos.

Na execução validada, o mapa de deduplicação teve 1070 registros. O script atualizou referências em aeroportos e circuitos e removeu as cidades duplicadas mapeadas.

5.4 Métricas após a deduplicação

A tabela `t1_metrics_after` registra as contagens finais depois da deduplicação. Isso permite comparar o estado da base antes e depois do tratamento.

6 Visões

O arquivo `04_views.sql` cria visões usadas para simplificar consultas e reaproveitar os conceitos trabalhados na disciplina.

6.1 Visões inspiradas no T1

6.1.1 View `vw_t1_pontos_pilotos_ano`

Mostra pontuação de pilotos por ano, usando a última rodada de cada temporada. Ela usa `driver_standings`, `standings`, `seasons` e `drivers`.

Essa view serve como base para consultas históricas de pontuação.

6.1.2 View `vw_t1_poles_por_piloto`

Conta quantas vezes cada piloto largou em primeiro lugar. Usa a tabela `results` e o filtro:

```
1 grid = 1
```

Ela retorna o piloto e a quantidade de poles.

6.1.3 View `vw_t1_paises_sede_corridas_cidades_aeroportos`

Lista países que sediam corridas, a quantidade de cidades e a quantidade de aeroportos nesses países. Ela usa `countries`, `cities`, `circuits` e `airports`.

6.2 Visões inspiradas no T4

6.2.1 Visão materializada Aeroportos_Brasileiros

Armazena fisicamente aeroportos vinculados a cidades brasileiras, incluindo dados do aeroporto, país, continente, cidade e população.

Foi feita como visão materializada porque a consulta envolve várias junções. A vantagem é acelerar consultas frequentes. A desvantagem é que ela pode ficar desatualizada se as tabelas de origem forem modificadas, exigindo `REFRESH MATERIALIZED VIEW`.

6.2.2 View Aeroportos_sem_cidades

Lista aeroportos sem cidade vinculada, ou seja, aeroportos com `city_id` nulo e coordenadas conhecidas.

6.2.3 View Cidades_brasileiras

Lista cidades brasileiras com população de pelo menos 100 mil habitantes e coordenadas conhecidas. Essa view segue a proposta do T4.

6.2.4 View Circuitos_completa

Lista circuitos com informações completas de localização: circuito, latitude, longitude, cidade, país, código do país e continente. Usa `LEFT JOIN` para manter circuitos mesmo quando alguma informação geográfica está ausente.

6.2.5 View Problemas_aeroportos

Identifica aeroportos sem cidade vinculada que estão a até 10 km de alguma cidade brasileira da view `Cidades_brasileiras`. Usa `earth_distance` e `ll_to_earth`.

Esses aeroportos aparecem nessa view porque têm `city_id` nulo, mas possuem coordenadas próximas de cidades brasileiras candidatas.

6.2.6 View Correcao_aeroportos

Lista os aeroportos vinculados aos casos encontrados em `Problemas_aeroportos`. Ela contém apenas dados da tabela `airports`, como pedido no exercício do T4.

6.3 Visões específicas do P4

6.3.1 View vw_p4_results_full

É a principal view auxiliar do P4. Ela reúne informações de:

- `results`;
- `racas`;

- seasons;
- circuits;
- drivers;
- constructors;
- countries;
- status.

Ela foi criada para evitar repetir a mesma sequência de junções em vários relatórios e dashboards.

Exemplo de uso:

```

1 SELECT driver_full_name, constructor_name, season_year, race_name
   , points
2 FROM vw_p4_results_full
3 WHERE season_year = 2025;
```

6.3.2 View vw_p4_constructor_driver_history

Lista pares distintos de escuderia e piloto que aparecem na tabela de resultados. Ela representa o histórico real de quais pilotos correram por quais escuderias.

Essa view é usada em consultas de escuderia, como a busca de pilotos por sobrenome e o relatório de quantidade de pilotos por escuderia.

6.3.3 View vw_p4_driver_latest_constructor

Identifica a escuderia mais recente de cada piloto. Para isso, usa `DISTINCT ON` e ordena por ano, data da corrida e rodada em ordem decrescente.

Ela é usada no dashboard do piloto para mostrar sua escuderia mais recente.

6.3.4 View vw_p4_cidades_brasileiras_todas

Lista todas as cidades brasileiras com coordenadas conhecidas. Diferente de `Cidades_brasileiras`, esta view não exige população mínima. Ela foi criada porque o Relatório 2 do P4 pede cidades brasileiras com o nome pesquisado, sem restringir por população.

6.3.5 View vw_p4_aeroportos_brasileiros_medium_large

Lista aeroportos brasileiros dos tipos `medium_airport` e `large_airport`, com coordenadas conhecidas. Essa view é usada pelo Relatório 2 do administrador.

7 Funções e procedimentos

O arquivo 05_functions.sql cria as funções e procedimentos usados pela aplicação.

7.1 Autenticação e log de acesso

7.1.1 Função p4_authenticate

Recebe login e senha e retorna os dados do usuário autenticado. Se a autenticação falhar, retorna zero linhas.

A função usa `crypt` para comparar a senha informada com o hash armazenado em `users.password`.

Exemplo:

```
1 SELECT * FROM p4_authenticate('admin', 'admin');
```

Retorno esperado no teste validado:

```
1 userid | login | tipo   | id_original | nome_exibicao
2 1      | admin | Admin |              | Administrador
```

7.1.2 Função p4_log_access

Registra eventos na tabela `users_log`. Aceita apenas LOGIN e LOGOUT.

Exemplo:

```
1 SELECT p4_log_access(1, 'LOGIN');
2 SELECT p4_log_access(1, 'LOGOUT');
```

7.2 Funções e procedimento inspirados no T2

7.2.1 Função Nome_Nacionalidade

Recebe o nome de uma escuderia e retorna a nacionalidade associada ao país da escuderia.

Exemplo:

```
1 SELECT Nome_Nacionalidade('Ferrari');
```

7.2.2 Função Pilotos_Nacionalidade

Recebe uma nacionalidade e percorre os pilotos com cursor, emitindo mensagens com `RAISE NOTICE`. Ela lança exceção se a nacionalidade for nula ou se nenhum piloto for encontrado.

7.2.3 Procedimento Cidade_Chamada

Recebe o nome de uma cidade, conta quantas cidades existem com esse nome e exibe cidade, população e país usando mensagens.

Exemplo:

```
1 CALL Cidade_Chamada('Sao Carlos');
```

7.2.4 Função Numero_Vitorias

Recebe nome, sobrenome e opcionalmente um ano. Retorna a quantidade de vitórias do piloto, considerando `position_order = 1`.

Exemplos:

```
1 SELECT Numero_Vitorias('Lewis', 'Hamilton');  
2 SELECT Numero_Vitorias('Sebastian', 'Vettel', 2018);
```

7.2.5 Função Pais_Continente

Usa cursor para retornar países com nomes de até 15 caracteres e seus continentes. Lança exceção se nenhum país for encontrado.

7.3 Dashboard do administrador

7.3.1 Função p4_admin_counts

Retorna três contagens:

- total de pilotos;
- total de escuderias;
- total de temporadas.

No teste validado, retornou 616 pilotos, 168 escuderias e 76 temporadas.

7.3.2 Função p4_admin_latest_races

Lista as corridas da temporada mais recente. Retorna temporada, corrida, circuito, data, horário e voltas registradas.

A quantidade de voltas é calculada com `MAX(laps)` por corrida, usando os resultados registrados.

7.3.3 Função p4_admin_latest_constructors_points

Lista escuderias que correram na temporada mais recente, com total de pontos somado a partir da view `vw_p4_results_full`.

7.3.4 Função `p4_admin_latest_drivers_points`

Lista pilotos que correram na temporada mais recente, sua escuderia mais recente e o total de pontos na temporada.

7.4 Dashboard da escuderia

7.4.1 Função `p4_constructor_dashboard`

Recebe o identificador da escuderia e retorna:

- nome da escuderia;
- quantidade de vitórias;
- quantidade de pilotos diferentes;
- primeiro ano com dados;
- último ano com dados.

A quantidade de vitórias usa `position_order = 1`. A quantidade de pilotos usa `COUNT(DISTINCT driver_id)`.

7.5 Dashboard do piloto

7.5.1 Função `p4_driver_dashboard`

Recebe o identificador do piloto e retorna informações agrupadas por ano e circuito:

- nome do piloto;
- escuderia mais recente;
- primeiro ano com dados;
- último ano com dados;
- ano;
- circuito;
- pontos;
- vitórias;
- total de corridas.

Essa função atende ao requisito do dashboard do piloto porque agrupa os dados por `season_year` e por circuito. Assim, se um piloto correu em vários circuitos no mesmo ano, cada circuito aparece separadamente.

Exemplo de consulta:

```
1 SELECT * FROM p4_driver_dashboard(1);
```

7.6 Relatórios do administrador

7.6.1 Relatório 1: função `p4_report_admin_status_count`

Retorna a quantidade de resultados por status.

Exemplo:

```
1 SELECT * FROM p4_report_admin_status_count();
```

No teste validado, alguns status mais frequentes foram `Finished`, `+1 Lap`, `Engine e +2 Laps`.

7.6.2 Relatório 2: função `p4_report_admin_airports_near_city`

Recebe o nome de uma cidade brasileira e retorna aeroportos brasileiros dos tipos `medium_airport` e `large_airport` localizados a no máximo 100 km.

A função usa:

- `vw_p4_cidades_brasileiras_todas`;
- `vw_p4_aeroportos_brasileiros_medium_large`;
- `earth_box`;
- `earth_distance`;
- `ll_to_earth`.

Exemplo:

```
1 SELECT * FROM p4_report_admin_airports_near_city('Sao Carlos');
```

No teste validado, foram encontrados aeroportos como Araraquara Airport, Campo Fontenelle e Leite Lopes Airport dentro do limite de distância.

7.6.3 Relatório 3: funções de escuderias e hierarquia de corridas

O Relatório 3 do administrador foi dividido em funções tabulares e também possui uma função agregadora em JSON.

A função `p4_report_admin_constructor_driver_count` lista todas as escuderias com a quantidade de pilotos que aparecem em seu histórico na tabela `results`.

A função `p4_report_admin_races_total` retorna o total geral de corridas cadastradas.

A função `p4_report_admin_races_by_circuit` retorna, por circuito:

- quantidade de corridas;
- mínimo de voltas registradas;
- média de voltas registradas;
- máximo de voltas registradas.

A função `p4_report_admin_race_details` retorna, para cada corrida:

- circuito;
- corrida;
- ano;
- voltas registradas;
- quantidade de pilotos participantes.

A função `p4_report_admin_3` reúne tudo em um objeto JSONB, com duas partes principais:

- `escuderias`;
- `hierarquia_corridas`.

A parte `hierarquia_corridas` contém:

- total geral de corridas;
- lista por circuito;
- lista por corrida.

Isso foi feito para facilitar a interface. O backend pode chamar a função agregadora ou chamar separadamente as funções tabulares, conforme for mais simples para montar a tela.

7.7 Relatórios da escuderia

7.7.1 Função `p4_report_constructor_driver_wins`

Recebe o identificador da escuderia e lista seus pilotos com a quantidade de vitórias de cada um. A vitória é identificada por `position_order = 1`.

7.7.2 Função `p4_report_constructor_status_count`

Recebe o identificador da escuderia e retorna a quantidade de resultados por status apenas no escopo daquela escuderia.

7.8 Relatórios do piloto

7.8.1 Função `p4_report_driver_points_by_year_race`

Recebe o identificador do piloto e lista as corridas em que ele obteve pontos. Retorna ano, corrida, circuito e pontos.

A função filtra apenas registros com pontos maiores que zero.

7.8.2 Função `p4_report_driver_status_count`

Recebe o identificador do piloto e retorna a quantidade de resultados por status nas corridas em que ele participou.

7.9 Funções de cadastro

7.9.1 Função `p4_create_constructor`

Insere uma nova escuderia em `constructors`. Recebe:

- `constructor_ref`;
- `name`;
- `country_id`;
- `wikipedia_url`.

A função retorna o identificador da escuderia criada. A criação do usuário correspondente é feita pelo gatilho de escuderias.

7.9.2 Função `p4_create_driver`

Insere um novo piloto em `drivers`. Antes da inserção, verifica se já existe piloto com o mesmo nome e sobrenome, desconsiderando diferenças de maiúsculas, minúsculas e espaços nas pontas.

Se já existir, lança exceção e cancela a inserção.

7.9.3 Função `p4_create_driver_for_constructor`

Recebe dados de um piloto e o identificador da escuderia logada. Ela chama `p4_create_driver` e, se a inserção for válida, registra a associação na tabela `constructor_drivers`.

Essa função é a parte SQL do fluxo de inserção de pilotos por arquivo. A leitura do arquivo não é feita pelo banco; ela deve ser feita pelo backend. O backend deve ler o arquivo linha por linha e chamar essa função para cada piloto.

7.9.4 Função `p4_constructor_find_driver_by_family_name`

Recebe o identificador da escuderia e o sobrenome pesquisado. Retorna pilotos com aquele sobrenome que já correram pela escuderia, usando a tabela `results`.

Retorna:

- nome completo;
- data de nascimento;
- país ou nacionalidade.

8 Usuários iniciais

O arquivo `06_seed_users.sql` cria os usuários iniciais da aplicação.

8.1 Administrador

Cria o usuário:

- login: `admin`;
- senha inicial: `admin`;
- tipo: `Admin`.

A senha é salva com hash usando `crypt` e `gen_salt`.

8.2 Escuderias

Para cada registro em `constructors`, cria um usuário com:

- login: `constructor_ref` seguido de `_c`;
- senha inicial: valor de `constructor_ref`;
- tipo: `Escuderia`;
- `id_original`: identificador da escuderia.

Exemplo ilustrativo: se `constructor_ref` for `mclaren`, o login será `mclaren_c` e a senha inicial será `mclaren`.

8.3 Pilotos

Para cada registro em `drivers`, cria um usuário com:

- login: `driver_ref` seguido de `_d`;
- senha inicial: valor de `driver_ref`;
- tipo: Piloto;
- `id_original`: identificador do piloto.

Exemplo ilustrativo: se `driver_ref` for `hamilton`, o login será `hamilton_d` e a senha inicial será `hamilton`.

Na execução validada, foram criados 1 usuário administrador, 168 usuários de escuderia e 616 usuários de piloto.

9 Gatilhos

O arquivo `07_triggers.sql` cria dois gatilhos principais.

9.1 Gatilho de pilotos

O gatilho `tr_p4_sync_driver_user` é executado após inserção ou atualização do campo `driver_ref` em `drivers`. Ele chama a função `p4_sync_driver_user`.

A função:

- verifica se `driver_ref` não está vazio;
- gera o login com o padrão `driver_ref` seguido de `_d`;
- gera a senha inicial a partir de `driver_ref`;
- salva a senha com hash;
- cria ou atualiza o usuário correspondente em `users`;
- cancela a operação se o login gerado já pertencer a outro usuário.

9.2 Gatilho de escuderias

O gatilho `tr_p4_sync_constructor_user` é executado após inserção ou atualização do campo `constructor_ref` em `constructors`. Ele chama a função `p4_sync_constructor_user`.

A função:

- verifica se `constructor_ref` não está vazio;

- gera o login com o padrão `constructor_ref` seguido de `_c`;
- gera a senha inicial a partir de `constructor_ref`;
- salva a senha com hash;
- cria ou atualiza o usuário correspondente em `users`;
- cancela a operação se o login gerado já pertencer a outro usuário.

Esses gatilhos garantem que o banco mantenha a tabela de usuários sincronizada automaticamente com pilotos e escuderias.

10 Índices

O arquivo `08_indices.sql` cria índices para melhorar o desempenho de junções, filtros e relatórios.

10.1 Índices de chaves estrangeiras

Foram criados índices em várias chaves estrangeiras, como:

- `countries.continent_id`;
- `cities.country_id`;
- `airports.city_id`;
- `circuits.city_id`;
- `racers.season_id`;
- `racers.circuit_id`;
- `results.race_id`;
- `results.driver_id`;
- `results.constructor_id`;
- `results.status_id`.

Esses índices ajudam porque as consultas do projeto usam muitas junções entre essas tabelas.

10.2 Índice de login

O índice `idx_p4_users_login` foi criado em `users(login)`. Ele auxilia a autenticação, pois a função `p4_authenticate` sempre busca o usuário pelo login.

10.3 Índice Hash de nome completo

O índice `idx_p4_drivers_nome_completo_hash` foi criado com Hash sobre a expressão do nome completo do piloto.

Ele foi inspirado no T5, em que a busca era feita por igualdade exata no nome completo.

Exemplo de consulta beneficiada:

```
1 SELECT given_name, family_name
2 FROM drivers
3 WHERE given_name || ' ' || family_name = 'Sebastian Vettel';
```

10.4 Índice B-tree parcial para cidades brasileiras

O índice `idx_p4_city_brazil_name_prefix` é criado apenas para cidades do Brasil. Ele usa `text_pattern_ops` para auxiliar buscas por prefixo no nome da cidade. Também usa `INCLUDE` para incluir latitude, longitude e população.

Esse índice foi inspirado no T5 e é útil para consultas que buscam cidades brasileiras pelo começo do nome.

10.5 Índices específicos para dashboards e relatórios

Foram criados índices sobre `results` para consultas por escuderia, piloto, corrida, status e posição:

- `idx_p4_results_constructor_position;`
- `idx_p4_results_constructor_status;`
- `idx_p4_results_driver_status;`
- `idx_p4_results_driver_race;`
- `idx_p4_results_constructor_driver.`

Eles auxiliam relatórios como:

- vitórias por piloto de uma escuderia;
- resultados por status de uma escuderia;
- pontos por corrida de um piloto;
- status por piloto.

10.6 Índices geográficos

Foram criados índices GiST usando `ll_to_earth` para cidades e aeroportos:

- `idx_p4_cities_brazil_ll_earth`;
- `idx_p4_airports_ll_earth`.

Eles auxiliam consultas com `earth_box` e `earth_distance`, especialmente o Relatório 2 do administrador.

11 Testes rápidos

O arquivo `09_testes_rapidos.sql` valida a base depois da criação.

Os testes executados incluem:

- autenticação do administrador;
- contagem de usuários por tipo;
- dashboard do administrador;
- corridas da temporada mais recente;
- relatório de status;
- relatório de aeroportos próximos de São Carlos;
- relatório 3 do administrador em seus três níveis;
- dashboard de escuderia;
- dashboard de piloto;
- relatório de vitórias por piloto de uma escuderia;
- relatório de pontos por ano e corrida de um piloto.

A execução validada terminou com a mensagem:

```
1 Base do P4 montada com sucesso.
```

Isso indica que todos os scripts foram executados na ordem correta e que as principais funções do banco responderam corretamente.

12 Como o backend deve usar o banco

A aplicação deve chamar as funções do banco em vez de repetir consultas grandes no código do backend.

12.1 Login

```
1 SELECT * FROM p4_authenticate($1, $2);
```

12.2 Log de acesso

```
1 SELECT p4_log_access($1, 'LOGIN');  
2 SELECT p4_log_access($1, 'LOGOUT');
```

12.3 Dashboards

```
1 SELECT * FROM p4_admin_counts();  
2 SELECT * FROM p4_admin_latest_races();  
3 SELECT * FROM p4_admin_latest_constructors_points();  
4 SELECT * FROM p4_admin_latest_drivers_points();  
5  
6 SELECT * FROM p4_constructor_dashboard($1);  
7  
8 SELECT * FROM p4_driver_dashboard($1);
```

12.4 Relatórios

```
1 SELECT * FROM p4_report_admin_status_count();  
2 SELECT * FROM p4_report_admin_airports_near_city($1);  
3 SELECT * FROM p4_report_admin_3();  
4  
5 SELECT * FROM p4_report_constructor_driver_wins($1);  
6 SELECT * FROM p4_report_constructor_status_count($1);  
7  
8 SELECT * FROM p4_report_driver_points_by_year_race($1);  
9 SELECT * FROM p4_report_driver_status_count($1);
```

12.5 Cadastro de pilotos por arquivo

A leitura do arquivo deve ser feita no backend, pois o usuário seleciona um arquivo do sistema operacional. Para cada linha do arquivo, o backend deve chamar:

```
1 SELECT p4_create_driver_for_constructor($1, $2, $3, $4, $5, $6);
```

A função SQL verifica duplicidade de nome e sobrenome por meio de `p4_create_driver`. Se o piloto for inserido, a associação com a escuderia é gravada em `constructor_drivers`.

13 Decisões de projeto

13.1 Schema já normalizado

O schema já foi criado com `country_id` em `drivers` e `constructors`. Isso evita criar colunas textuais de nacionalidade para depois removê-las. A base final fica mais limpa e com menos redundância.

13.2 Carga com staging

O uso do schema `stg` foi escolhido para tornar a carga mais segura. Os dados entram primeiro como texto e só depois são convertidos para os tipos finais.

13.3 Deduplicação conservadora

A deduplicação de cidades usa mesmo país e mesmo nome normalizado. Esse critério foi escolhido para evitar correções agressivas que poderiam unir cidades diferentes por engano.

13.4 Views para reduzir repetição

A view `vw_p4_results_full` evita repetir várias junções em todos os relatórios. Isso deixa as funções menores e mais fáceis de explicar.

13.5 Funções para centralizar regras

Regras como autenticação, dashboards, relatórios e cadastros foram colocadas em funções SQL. Isso facilita o backend e deixa os comandos SQL explícitos.

13.6 Gatilhos para manter usuários sincronizados

Os gatilhos garantem que novos pilotos e novas escuderias gerem automaticamente usuários da aplicação. Assim, essa regra não depende apenas do backend.

13.7 Índices voltados às consultas do projeto

Os índices foram criados com base nos filtros e junções usados de fato no projeto: login, piloto, escuderia, status, corrida, cidades e aeroportos.

14 Resumo dos objetos existentes

14.1 Tabelas

- `seasons`, `continents`, `time_zones`, `language_names`, `feature_codes`, `countries`, `iso_language_codes`, `country_languages`, `cities`;

- airport_types, airports;
- circuits, constructors, drivers, status, standings, races, qualifying, results, driver_standings, constructor_standings;
- users, users_log, constructor_drivers;
- tabelas auxiliares de métricas: t1_metrics_before, t1_country_reference_metrics, t1_city_dedup_map, t1_metrics_after.

14.2 Views e visão materializada

- vw_t1_pontos_pilotos_ano;
- vw_t1_poles_por_piloto;
- vw_t1_paises_sede_corridas_cidades_aeroportos;
- Aeroportos_Brasileiros;
- Aeroportos_sem_cidades;
- Cidades_brasileiras;
- Circuitos_completa;
- Problemas_aeroportos;
- Correcao_aeroportos;
- vw_p4_results_full;
- vw_p4_constructor_driver_history;
- vw_p4_driver_latest_constructor;
- vw_p4_cidades_brasileiras_todas;
- vw_p4_aeroportos_brasileiros_medium_large.

14.3 Funções e procedimentos

- autenticação e log: p4_authenticate, p4_log_access;
- T2: Nome_Nacionalidade, Pilotos_Nacionalidade, Cidade Chamada, Numero_Vitorias, Pais_Continente;
- dashboards: p4_admin_counts, p4_admin_latest_races, p4_admin_latest_constructors_points, p4_admin_latest_drivers_points, p4_constructor_dashboard, p4_driver_dashboard;

- relatórios do admin: p4_report_admin_status_count, p4_report_admin_airports_near_city, p4_report_admin_constructor_driver_count, p4_report_admin_races_total, p4_report_admin_races_by_circuit, p4_report_admin_race_details, p4_report_admin_3;
- relatórios de escuderia: p4_report_constructor_driver_wins, p4_report_constructor_status_count;
- relatórios de piloto: p4_report_driver_points_by_year_race, p4_report_driver_status_count;
- cadastros e consulta: p4_create_constructor, p4_create_driver, p4_create_driver_for_constructor, p4_constructor_find_driver_by_family_name;
- funções de gatilho: p4_sync_driver_user, p4_sync_constructor_user.

14.4 Gatilhos

- tr_p4_sync_driver_user;
- tr_p4_sync_constructor_user.

14.5 Índices principais

- índices de chaves estrangeiras em países, cidades, aeroportos, circuitos, corridas, classificações e resultados;
- idx_p4_users_login;
- idx_p4_drivers_nome_completo_hash;
- idx_p4_city_brazil_name_prefix;
- idx_p4_results_constructor_position;
- idx_p4_results_constructor_status;
- idx_p4_results_driver_status;
- idx_p4_results_driver_race;
- idx_p4_results_constructor_driver;
- idx_p4_races_season_circuit;
- idx_p4_airports_city_type;
- idx_p4_drivers_family_name_lower;
- idx_p4_cities_brazil_ll_earth;
- idx_p4_airports_ll_earth.

15 Conclusão

A parte SQL do projeto foi organizada para atender aos requisitos do P4 e para reaproveitar os conceitos trabalhados na disciplina.

O banco contém tabelas normalizadas, carga robusta por staging, deduplicação conservadora, visões, visão materializada, funções, procedimentos, gatilhos, índices, usuários, logs de acesso, dashboards e relatórios.

A execução dos testes rápidos confirmou que a base foi montada com sucesso e que os principais objetos estão funcionando corretamente.